

Temporización No Bloqueante con millis(): Control Concurrente de Múltiples LEDs

Reporte de Práctica — Sistemas Programables

1. Descripción del proyecto

Este proyecto tiene como objetivo solucionar la problemática planteada por el antipatrón de `delay()`, implementando una temporización no bloqueante mediante la función **millis()**. Se plantea un circuito con tres LEDs para que parpadeen de forma verdaderamente independiente y simultánea a intervalos de 500 ms (LED1), 1000 ms (LED2) y 1500 ms (LED3).

A diferencia del enfoque bloqueante, el programa utiliza un contador de milisegundos para monitorear el tiempo transcurrido sin detener el flujo principal de ejecución, permitiendo multitarea cooperativa en el microcontrolador.

2. Objetivos de aprendizaje

- Comprender el funcionamiento de la función `millis()` como reloj interno en Arduino.
- Implementar el patrón de verificación de intervalos de tiempo para evitar el uso de funciones bloqueantes.
- Lograr la ejecución simultánea e independiente de múltiples tareas en el microcontrolador.
- Comparar el rendimiento y comportamiento del sistema frente al enfoque secuencial con `delay()`.

3. Material utilizado

- Arduino UNO R4 WiFi
- Protoboard
- 3 LEDs de colores distintos
- 3 resistencias limitadoras de corriente (220 Ω)
- Cables de conexión (jumpers)
- Cable USB para alimentación y programación

4. Diagrama de conexión

La conexión física se mantiene idéntica a la práctica anterior: los LEDs se conectan a los pines digitales 2, 3 y 4 del Arduino UNO R4 WiFi, cada uno en serie con su respectiva resistencia limitadora conectada a GND.

5. Análisis del código: Temporización con millis()

El código declara marcas de tiempo (variables `unsigned long`) para guardar la última vez que cambió el estado de cada LED. En cada iteración del ciclo `loop()`, se evalúa la diferencia entre el tiempo actual transcurrido y la marca de tiempo anterior:

```
const int LED1 = 2;
const int LED2 = 3;
const int LED3 = 4;

unsigned long tiempoPrevio1 = 0;
```

```

unsigned long tiempoPrevio2 = 0;
unsigned long tiempoPrevio3 = 0;

const long intervalo1 = 500;
const long intervalo2 = 1000;
const long intervalo3 = 1500;

int estadoLED1 = LOW;
int estadoLED2 = LOW;
int estadoLED3 = LOW;

void setup() {
  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);
  pinMode(LED3, OUTPUT);
}

void loop() {
  unsigned long tiempoActual = millis();

  // Control independiente de LED 1 (500 ms)
  if (tiempoActual - tiempoPrevio1 >= intervalo1) {
    tiempoPrevio1 = tiempoActual;
    estadoLED1 = !estadoLED1;
    digitalWrite(LED1, estadoLED1);
  }

  // Control independiente de LED 2 (1000 ms)
  if (tiempoActual - tiempoPrevio2 >= intervalo2) {
    tiempoPrevio2 = tiempoActual;
    estadoLED2 = !estadoLED2;
    digitalWrite(LED2, estadoLED2);
  }

  // Control independiente de LED 3 (1500 ms)
  if (tiempoActual - tiempoPrevio3 >= intervalo3) {
    tiempoPrevio3 = tiempoActual;
    estadoLED3 = !estadoLED3;
    digitalWrite(LED3, estadoLED3);
  }
}

```

Gracias a esta estructura, el procesador evalúa continuamente los tres LEDs sin quedarse detenido en ningún momento. Cada evento ocurre de forma precisa y concurrente.

6. Conclusiones

El uso de `millis()` resuelve completamente la limitación del `delay()`. Permite que múltiples tareas con distintos tiempos de temporización convivan en el mismo programa sin interferir entre sí. Esta metodología es esencial para el desarrollo de proyectos reales en sistemas programables, donde se requiere atender sensores, comunicaciones y actuadores de forma simultánea.

Anexo. Evidencias de la práctica

Control de LEDs con millis()

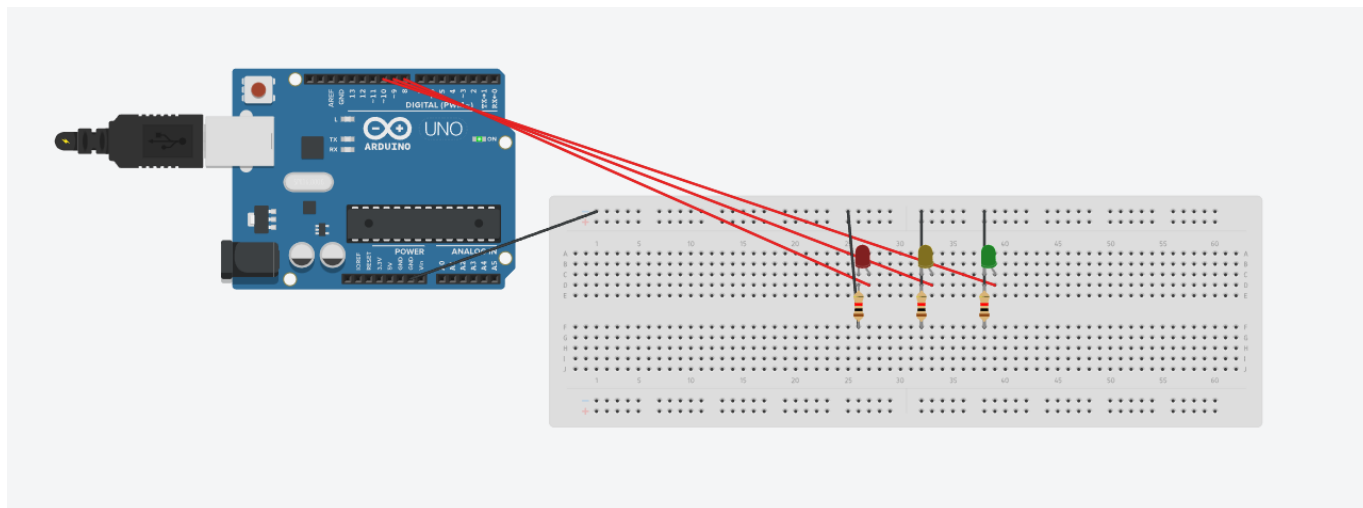


Figura 1. Diagrama del circuito de tres LEDs utilizado para comparar la temporización con delay() y millis()).

Archivo: Diagrama Leds.png

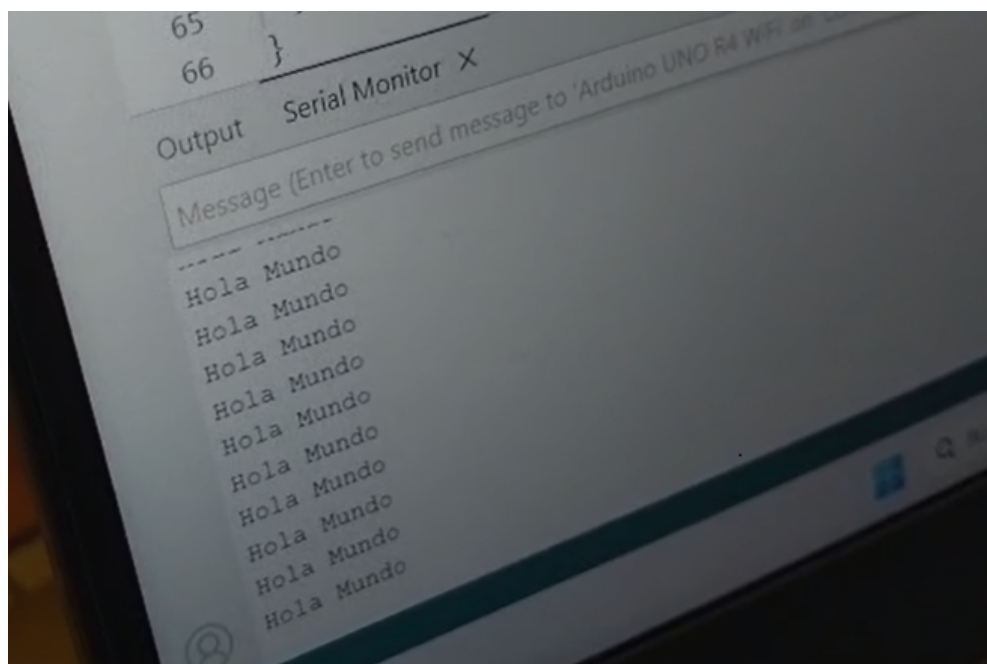


Figura 2. Monitor Serial con mensajes «Hola Mundo», evidencia disponible de la ejecución de la práctica con millis()).

Archivo: Leds Milic.png